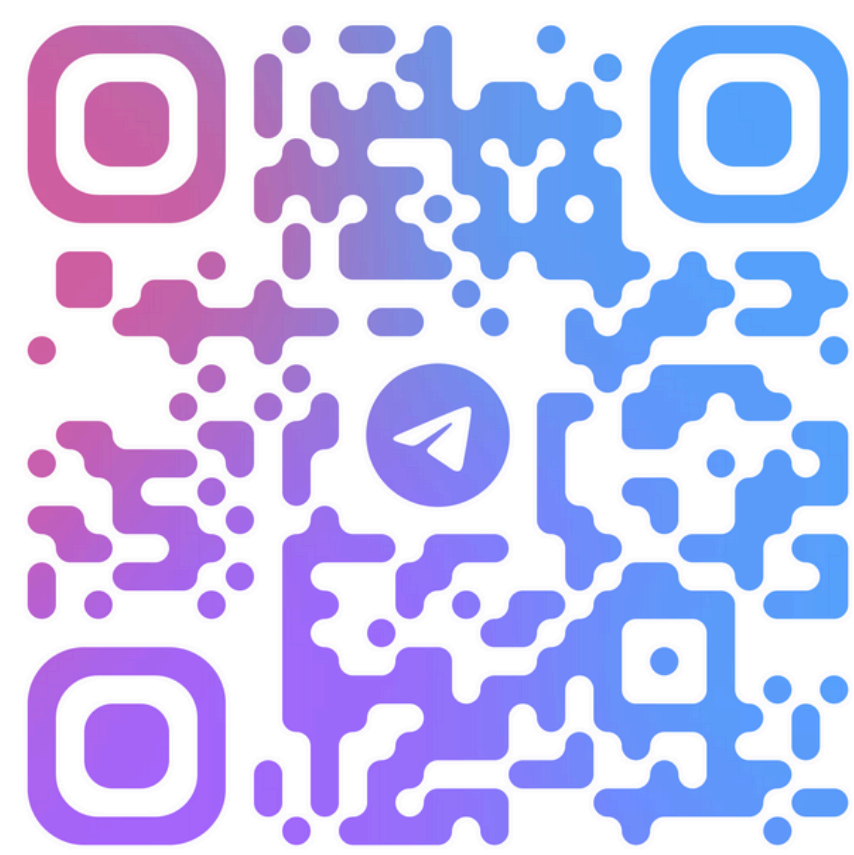




@RUSLANSENATOROV



МАТЕМАТИКА ДЛЯ DATA SCIENCE

```
class MatrixLinearRegression:

    def fit(self, X, y):
        X = np.insert(X, 0, 1, axis=1)    # add ones vector
        XT_X_inv = np.linalg.inv(X.T @ X)    # (X.T * X) ** (-1) inverse matrix
        weights = np.linalg.multi_dot([XT_X_inv, X.T, y])    # XT_X_inv * X.T * y
        self.bias, self.weights = weights[0], weights[1:]

    def predict(self, X_test):
        return X_test @ self.weights + self.bias
```

```
df_path = 'data.csv'

income = pd.read_csv(df_path)
X1, y1 = income.iloc[:, :-1].values, income.iloc[:, -1].values

X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, random_state=0)

income.corr()
```

.corr() вычисляет коэффициенты корреляции между столбцами.

```
sk_linear_regression = LinearRegression()
sk_linear_regression.fit(X1_train, y1_train)
```

# age	# experience	# income
25	1	30450
30	3	35670
47	2	31580
32	5	40130
43	10	47830
51	7	41630
28	5	41340
33	4	37650
37	5	40250
39	8	45150
29	1	27840
47	9	46110
54	5	36720
51	4	34800
44	12	51300
41	6	38900
58	17	63600
23	1	30870
44	9	44190
37	10	48700

Разделение на обучающую и тестовую выборки (без масштабирования):
75% — обучение, 25% — тест (по умолчанию)

Позволяет сравнивать обучение на "сырых" и масштабированных данных.

Данные разбиваются случайно,
но в один и тот же запуск скрипта результат может меняться (если не задан random_state)

```
train_test_split(X, y, test_size=0.25, random_state=42)
```

# age	# experience	# income
25	1	30450
30	3	35670
47	2	31580
32	5	40130
43	10	47830
51	7	41630
28	5	41340
33	4	37650
37	5	40250
39	8	45150
29	1	27840
47	0	46110
54	5	36720
51	4	34800
44	12	51300
41	6	38900
58	17	63600
23	1	30870
44	9	44190
37	10	48700

```
def train_test_split(  
    *arrays,  
    test_size=None,  
    train_size=None,  
    random_state=None,  
    shuffle=True,  
    stratify=None,  
):  
    """Split arrays or matrices into random train and test subsets.  
  
    Quick utility that wraps input validation,  
    ``next(ShuffleSplit().split(X, y))``, and application to input data  
    into a single call for splitting (and optionally subsampling) data into a  
    one-liner.  
  
    Read more in the :ref:`User Guide <cross_validation>`.  
  
    Parameters  
    -----  
    *arrays : sequence of indexables with same length / shape[0]  
        Allowed inputs are lists, numpy arrays, scipy-sparse  
        matrices or pandas dataframes.  
  
    test_size : float or int, default=None  
        If float, should be between 0.0 and 1.0 and represent the proportion  
        of the dataset to include in the test split. If int, represents the  
        absolute number of test samples. If None, the value is set to the  
        complement of the train size. If ``train_size`` is also None, it will  
        be set to 0.25.  
  
    train_size : float or int, default=None  
        If float, should be between 0.0 and 1.0 and represent the  
        proportion of the dataset to include in the train split. If  
        int, represents the absolute number of train samples. If None,  
        the value is automatically set to the complement of the test size.
```


X — матрица признаков размера

```
X = np.insert(X, 0, 1, axis=1)
```

Nº	Bias (1)	Площадь (м²) x_1	Кол-во комнат x_2	Цена y (тыс. \$)
1	1	50	2	160
2	1	60	3	190
3	1	70	3	220
4	1	80	4	250
5	1	90	5	280

Добавлять колонку с 1 в csv не нужно. Это делается внутри библиотеки, на уровне кода

	axis = 1			
	Column 1	Column 2	Column 3	Column 4
Row 1				
Row 2				

axis = 0 (vertical arrow)

axis = 1 (horizontal arrow)

→ затем.

Вставляем в модель свободный

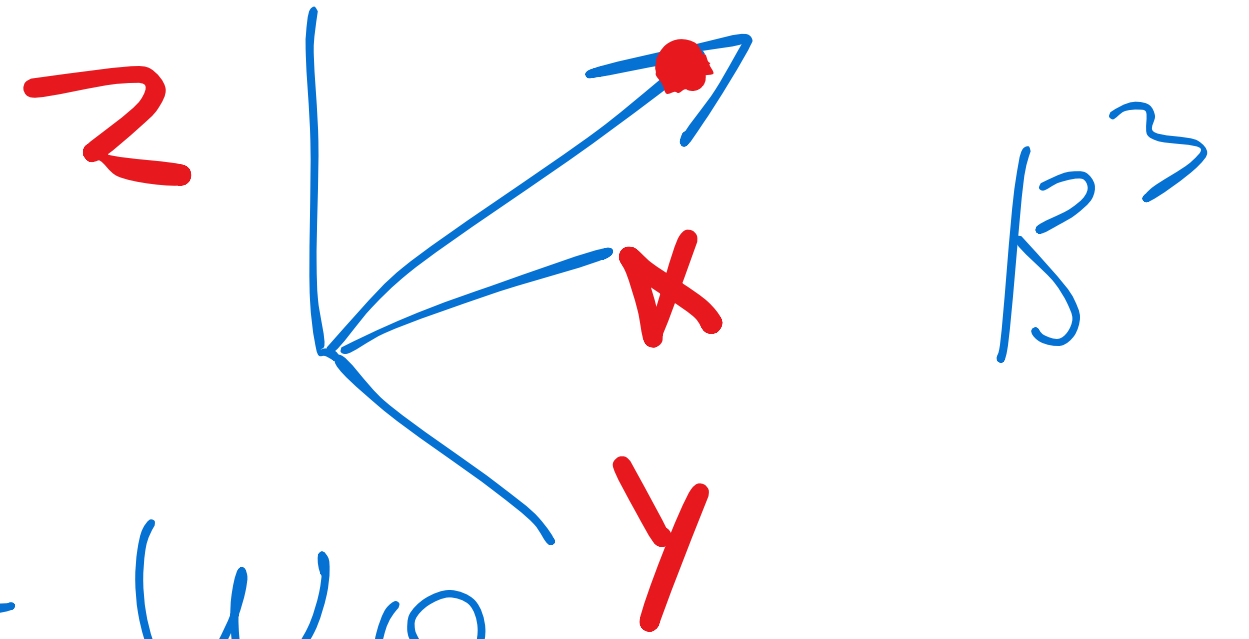
член (insert) / смещение

$$y = w_0 + w_1 x_1 + w_2 x_2$$

Чтобы упростить вычисления и представление модели в матричной форме, мы добавляем фиктивный признак, который всегда равен 1

$$f_w(x_i) = \langle w, x_i \rangle + w_0$$


```
X = np.insert(X, 0, 1, axis=1)
```

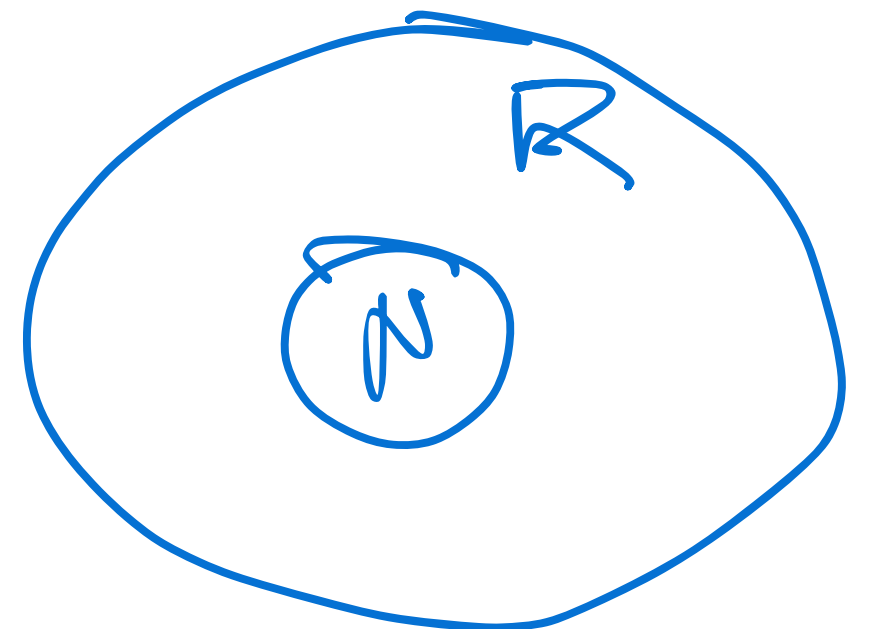


$$f_w(x_i) = \langle w, x_i \rangle + w_0$$

Ity crms (x, y) 2ge $y = (y_i)_{i=1}^N \in \mathbb{R}^N$

$$X = (x_i)_{i=1}^N \in \mathbb{R}^{N \times D}$$

$x_i \in \mathbb{R}^D$ D - количество признаков



$$(x_{i1} \dots x_{iD}) \cdot \begin{pmatrix} w_1 \\ \vdots \\ w_D \end{pmatrix} + w_0 =$$

$$(1 \ x_{i1} \dots x_{iD}) \cdot \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_D \end{pmatrix}$$

$$f_w(x_i) = \langle w, x_i \rangle$$

$$\langle w, x_i \rangle = \sum_{j=1}^n w_j x_{ij}$$

	log	exp	
1	20	7	100
2			
3			
4			

$$x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \in \mathbb{R}^{3 \times 1}$$

$$w = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} \in \mathbb{R}^{3 \times 1}$$

$$(3 \times 1) / (3 \times 1)$$


$$w = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}^T = (w_1, w_2, w_3) \in \mathbb{R}^{1 \times 3}$$

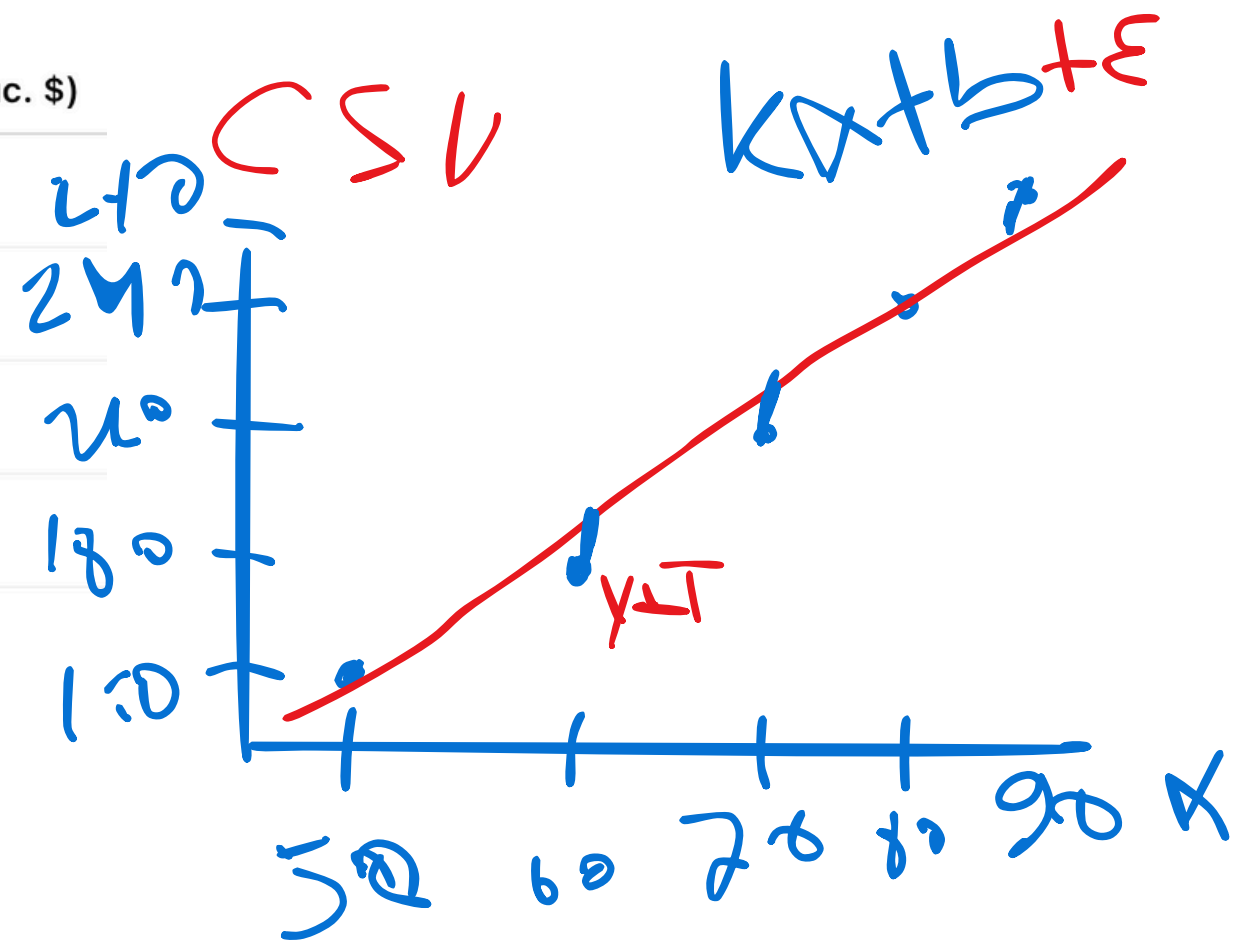
$$w^T x = (w_1^{1} \ w_2^{2} \ w_3^{3}) \cdot \begin{pmatrix} x_1^5 \\ x_2^6 \\ x_3^7 \end{pmatrix} \rightarrow$$

$$w_1 x_1 + w_2 x_2 + w_3 x_3 = 5$$

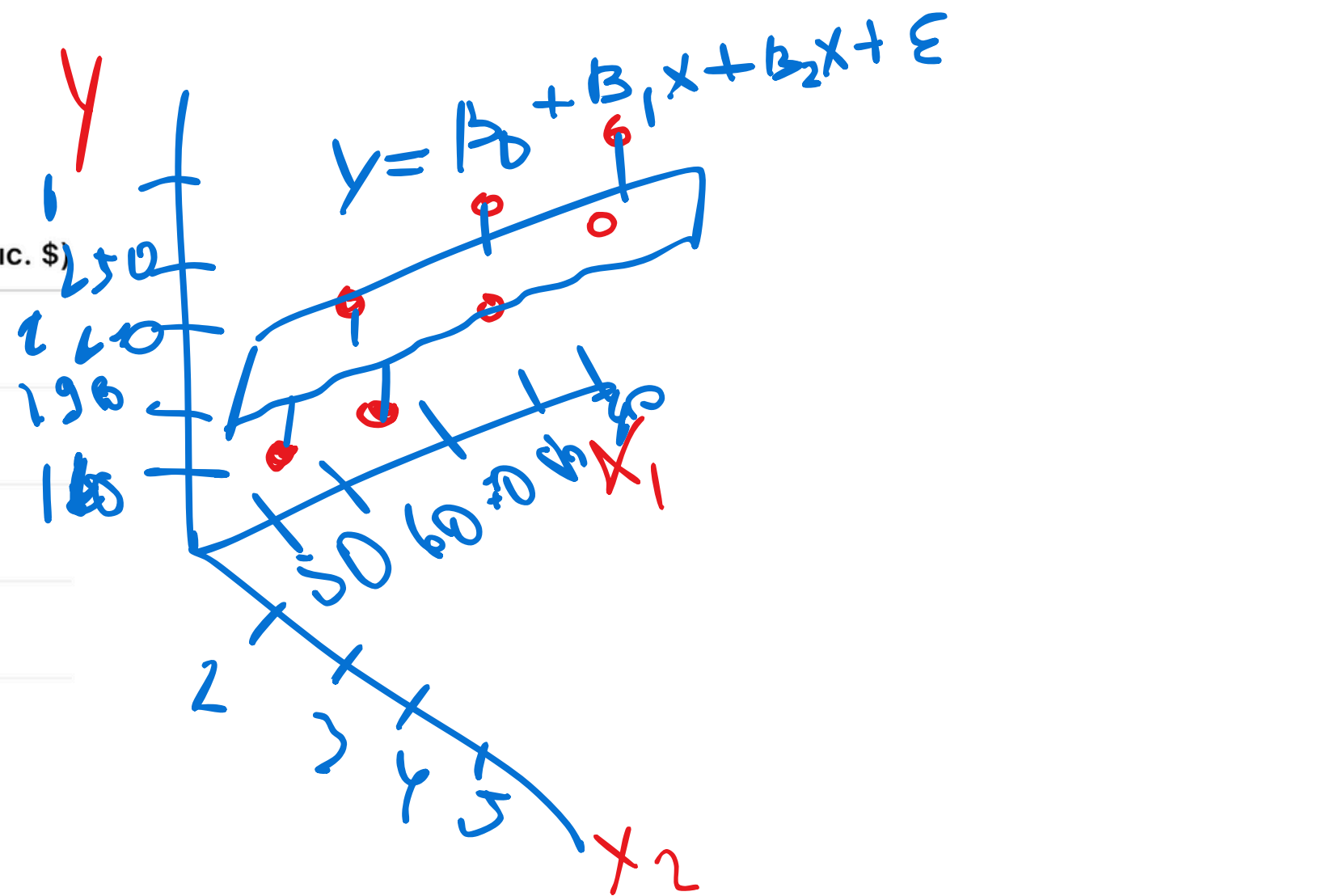
$$x w^T = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \cdot (w_1 \ w_2 \ w_3) = \begin{pmatrix} x_1 w_1 & x_1 w_2 & x_1 w_3 \\ x_2 w_1 & x_2 w_2 & x_2 w_3 \\ x_3 w_1 & x_3 w_2 & x_3 w_3 \end{pmatrix} \in \mathbb{R}^{3 \times 3}$$

```
XT_X_inv = np.linalg.inv(X.T @ X)    #  $(X.T * X)^{-1}$  inverse matrix  
weights = np.linalg.multi_dot([XT_X_inv, X.T, y])  #  $XT\_X\_inv * X.T * y$ 
```

№	Площадь (м²) x	Цена дома y (тыс. \$)
1	50	150
2	60	180
3	70	210
4	80	240
5	90	270



№	Bias (1)	Площадь (м²) x_1	Кол-во комнат x_2	Цена y (тыс. \$)
1		50	2	160
2		60	3	190
3		70	3	220
4		80	4	250
5		90	5	280



◆ 1. Скалярная форма (поэлементная)

Записывается как сумма произведений каждого признака на свой коэффициент:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p$$

Эта форма **удобна для понимания**, особенно при малом числе признаков. Используется в теории, формулах, объяснениях.

$$\sum x_i \theta_i = \theta_0 \cancel{x_0} + \theta_1 x_1 + \dots + \theta_p x_p$$

1

◆ 2. Векторная форма

Записывается через скалярное произведение двух векторов:

$$y = \theta^T x = \sum_{j=0}^p \theta_j x_j$$

где:

- $\theta = [\theta_0, \theta_1, \dots, \theta_p]^T$ — вектор параметров,
- $x = [1, x_1, \dots, x_p]^T$ — вектор признаков (с первым элементом 1 — для свободного члена).

✚ Здесь y — одно предсказание для одного объекта.

📊 Матричная форма

Если у нас есть n наблюдений, и каждый объект имеет p признаков:

- $X \in \mathbb{R}^{n \times (p+1)}$ — матрица регрессоров с первым столбцом из единиц (для θ_0),
- $\theta \in \mathbb{R}^{(p+1) \times 1}$ — вектор коэффициентов,
- $y \in \mathbb{R}^{n \times 1}$ — вектор ответов.

$$y = X\theta$$

🧠 Решение через нормальное уравнение

Чтобы найти θ , решаем:

$$\theta = (X^T X)^{-1} X^T y$$

◆ 4. Функциональная форма

Иногда записывают как:

$$f(x) = \theta^T x$$

или

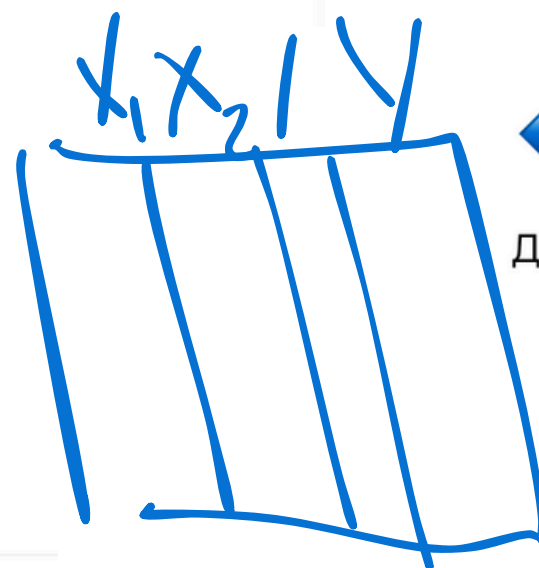
$$\hat{y} = f(x; \theta)$$

где явно показывают, что модель — это **функция от параметров и входа**.

◆ 5. Система уравнений (развёрнутая форма)

Для наглядности можно записать как систему:

$$\begin{cases} \hat{y}^{(1)} = \theta_0 + \theta_1 x_1^{(1)} + \dots + \theta_p x_p^{(1)} \\ \hat{y}^{(2)} = \theta_0 + \theta_1 x_1^{(2)} + \dots + \theta_p x_p^{(2)} \\ \vdots \\ \hat{y}^{(n)} = \theta_0 + \theta_1 x_1^{(n)} + \dots + \theta_p x_p^{(n)} \end{cases}$$



- $\theta \in \mathbb{R}^{(p+1) \times 1}$ — **вектор-столбец**,
то
- $\theta^T \in \mathbb{R}^{1 \times (p+1)}$ — **вектор-строка**.

📌 Теперь главное:

Мы хотим вычислить **скалярное произведение** двух векторов:

$$f(x) = \theta_0 \cdot x_0 + \theta_1 \cdot x_1 + \dots + \theta_p \cdot x_p$$

То есть:

вектор-строка θ^T умножается на **вектор-столбец** x :

$$f(x) = \theta^T x$$

📐 Размерности:

Если:

- $\theta^T \in \mathbb{R}^{1 \times (p+1)}$
- $x \in \mathbb{R}^{(p+1) \times 1}$

то:

$$\theta^T x \in \mathbb{R}^{1 \times 1} \quad (\text{скаляр — результат предсказания})$$

🧠 Что будет без транспонирования?

Если бы ты написал просто θx , то это матричное произведение **двух векторов-столбцов**, и результат — **матрица**, а не скаляр:

$$\theta \in \mathbb{R}^{(p+1) \times 1}, \quad x \in \mathbb{R}^{(p+1) \times 1} \Rightarrow \text{Ошибка умножения (размерности не совпадают)}$$

📌 Вывод:

$$f(x) = \theta^T x$$

— это **скалярное произведение**, дающее **одно число** — **предсказание модели**.

Транспонирование θ^T нужно, чтобы размерности совпали и получилось правильное умножение.

Пусть у нас:

- p — количество признаков (без учёта свободного члена),
- k — количество целевых переменных (выходов),
- $\Theta \in \mathbb{R}^{(p+1) \times k}$ — матрица весов.

Пример:

Допустим, $p = 2, k = 3$, тогда:

♦ Матрица признаков $x \in \mathbb{R}^{3 \times 1}$:

$$x = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}$$

(добавили 1 для свободного члена θ_0)

♦ Матрица параметров $\Theta \in \mathbb{R}^{3 \times 3}$:

$$\Theta = \begin{bmatrix} \theta_{00} & \theta_{01} & \theta_{02} \\ \theta_{10} & \theta_{11} & \theta_{12} \\ \theta_{20} & \theta_{21} & \theta_{22} \end{bmatrix}$$

Где:

- Столбец 0: веса для 1-й выходной переменной,
- Столбец 1: веса для 2-й переменной,
- Столбец 2: веса для 3-й переменной.



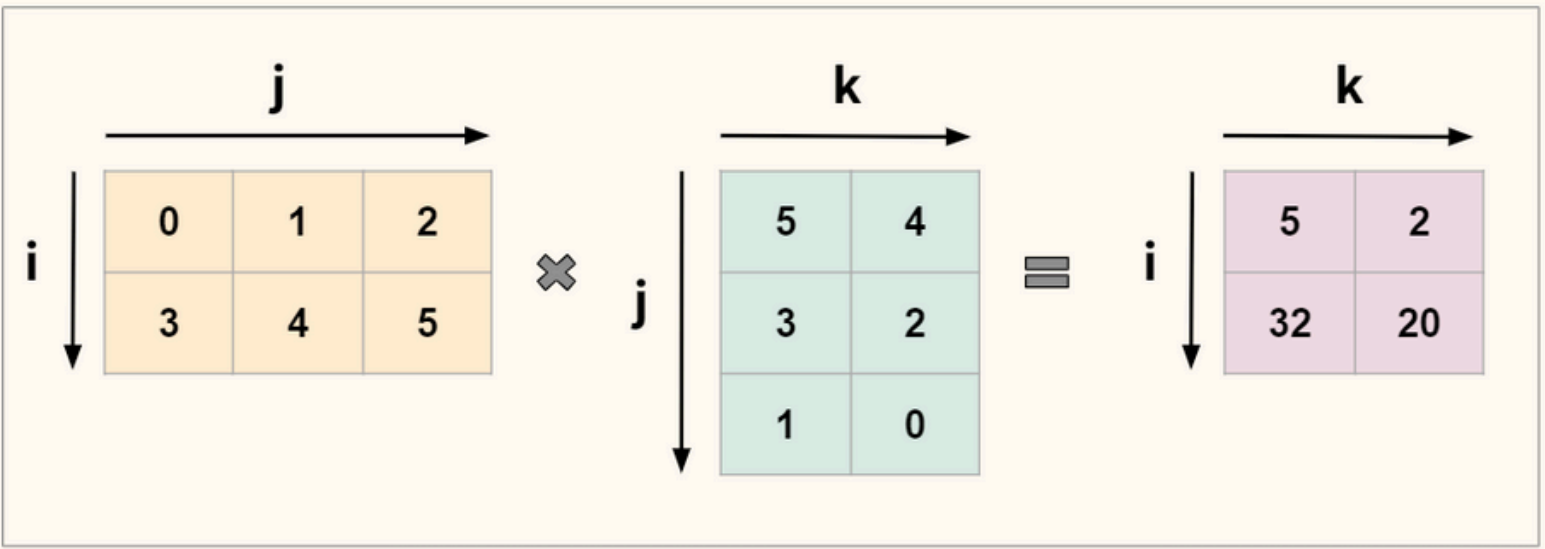
♦ Предсказание:

$$\hat{y} = x^T \Theta = \begin{bmatrix} 1 & x_1 & x_2 \end{bmatrix} \cdot \begin{bmatrix} \theta_{00} & \theta_{01} & \theta_{02} \\ \theta_{10} & \theta_{11} & \theta_{12} \\ \theta_{20} & \theta_{21} & \theta_{22} \end{bmatrix} = \begin{bmatrix} \hat{y}_1 & \hat{y}_2 & \hat{y}_3 \end{bmatrix}$$

1x3

✓ Вывод:

- Θ — это матрица
- Вектор x умножа



Пусть у нас:

- **2 признака** (например: площадь и возраст),
- **3 целевых переменных** (например: цена, аренда, налог).

Тогда:

- $x \in \mathbb{R}^{3 \times 1}$ (добавим единицу для свободного члена),
 - $\Theta \in \mathbb{R}^{3 \times 3}$ — веса для трёх регрессионных моделей.
-

 **Входной вектор:**

$$x = \begin{bmatrix} 1 \\ 50 \\ 10 \end{bmatrix} \quad (1 \text{ — свободный член, } 50 \text{ — площадь, } 10 \text{ — возраст})$$

 **Матрица параметров Θ :**

$$\Theta = \begin{bmatrix} 10000 & 5000 & 2000 \\ 300 & 150 & 100 \\ -50 & -30 & -10 \end{bmatrix}$$

Каждый столбец — коэффициенты для одного из выходов:

- Столбец 1: модель для **цены**
- Столбец 2: модель для **аренды**
- Столбец 3: модель для **налога**

 Вычислим предсказание:

$$\hat{y} = x^T \cdot \Theta$$

Сначала транспонируем x :

$$x^T = [1 \quad 50 \quad 10]$$

Теперь умножим:

$$\hat{y} = [1 \quad 50 \quad 10] \cdot \begin{bmatrix} 10000 & 5000 & 2000 \\ 300 & 150 & 100 \\ -50 & -30 & -10 \end{bmatrix} \approx [24500 \quad 12200 \quad 6900]$$

 **Умножаем по столбцам:**

$$1 \cdot 10000 + 50 \cdot 300 + 10 \cdot (-50) = 24500$$

Переменная 1 (цена):

$$1 \cdot 10000 + 50 \cdot 300 + 10 \cdot (-50) = 10000 + 15000 - 500 = 24500$$

Переменная 2 (аренда):

$$1 \cdot 5000 + 50 \cdot 150 + 10 \cdot (-30) = 5000 + 7500 - 300 = 12200$$

Переменная 3 (налог):

$$1 \cdot 2000 + 50 \cdot 100 + 10 \cdot (-10) = 2000 + 5000 - 100 = 6900$$

 **Ответ:**

$$\hat{y} = [24500 \quad 12200 \quad 6900]$$

Это означает:

при $x = [1, 50, 10]$ модель предсказывает:

- Цена: **24500**
- Аренда: **12200**
- Налог: **6900**

Нормальное уравнение

```
XT_X_inv = np.linalg.inv(X.T @ X) # (X.T * X) ** (-1) inverse matrix
weights = np.linalg.multi_dot([XT_X_inv, X.T, y]) # XT_X_inv * X.T * y
```

```
theta = np.linalg.inv(X.T @ X) @ X.T @ y
```

$$\theta = (X^T X)^{-1} X^T y$$

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p$$

уравнения. Подходит, если:

- данные не слишком большие (иначе вычисление обратной матрицы дорогое),
- матрица $X^T X$ не вырожденная (иначе нужно использовать псевдообратную).

— квадратная матрица, определитель которой отличен от нуля

$$\begin{matrix} & K & & \\ M & \boxed{A_{ik}} & * & \boxed{B_{kj}} & = & \boxed{C_{ji}} & \\ & & & N & & \end{matrix}$$

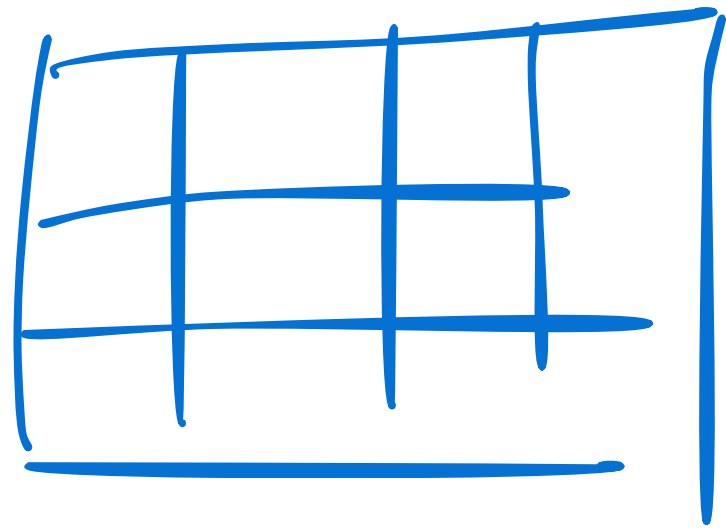
Handwritten red annotations: "20" above K, "2" above N, "2" below M, "20" below the asterisk, and "2" below the final box.

#	age	#	experience	#	income
25		1		30450	
30		3		35670	
47		2		31580	
32		5		40130	
43		10		47830	
51		7		41630	
28		5		41340	
33		4		37650	
37		5		40250	
39		8		45150	
29		1		27840	
47		9		46110	
54		5		36720	
51		4		34800	
44		12		51300	
41		6		38900	
58		17		63600	
23		1		30870	
44		9		44190	
37		10		48700	

2

```
XT_X_inv = np.linalg.inv(X.T @ X)
```

- 1) Детерминант (определитель) $\neq 0$
- 2) Ранг матрицы равен числу CC (числу строк)
- 3) Матрица не имеет обратных



$$A^{-1} = \frac{1}{|A|} \times A_{ij}^T$$


```
XT_X_inv = np.linalg.inv(X.T @ X)
```

$$X \approx \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

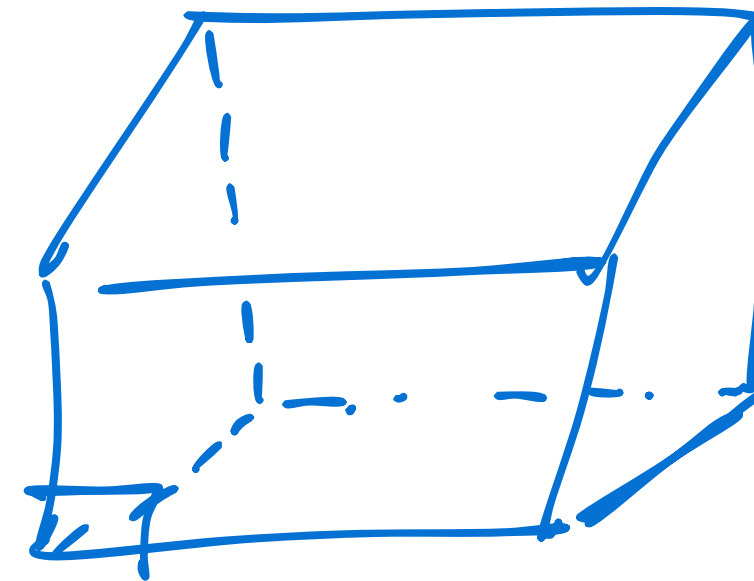
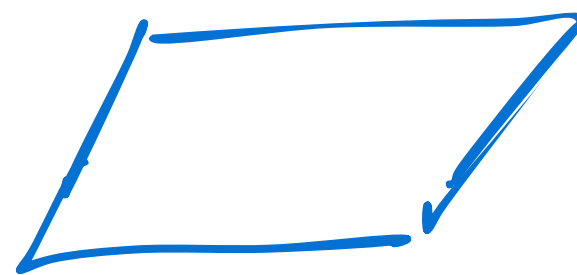
$$Y \approx \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

X_0	X_1	Y
1	1	2
1	2	3

$$X^T \approx \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$X^T X \approx \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \approx \begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix}$$

$$A^{-1} = \frac{1}{|A|} \times A_{ij}^T$$



Геометрическая интерпретация:

Для матрицы 2x2, определитель представляет собой (со знаком) площадь параллелограмма, образованного векторами-столбцами (или векторами-строками).

Для матрицы 3x3, определитель – это (со знаком) объем параллелепипеда, образованного векторами-столбцами.

```
XT_X_inv = np.linalg.inv(X.T @ X)
```

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$A = \begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix} \Rightarrow 2 \cdot 5 - (3 \cdot 3) = 1$$

$$A^{-1} = 1 \cdot \begin{bmatrix} 5 & -3 \\ -3 & 2 \end{bmatrix}$$

$$\begin{bmatrix} 5 & -3 \\ -3 & 2 \end{bmatrix}$$

$$\begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix} =$$

нахождению обратной матрицы

$$M = \begin{pmatrix} 5 & -3 \\ -3 & 2 \end{pmatrix} \Rightarrow \begin{pmatrix} 5 & -3 \\ -3 & 2 \end{pmatrix}$$

$$A^{-1}A = E \Rightarrow \begin{bmatrix} 5 & -3 \\ -3 & 2 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 3 & 5 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

```
weights = np.linalg.multi_dot([XT_X_inv, X.T, y])
```

$$\begin{bmatrix} 5 & -1 \\ -1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}$$

$$\begin{bmatrix} 5 & -1 \\ -1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 5 \\ 8 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$\oplus \rightarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ weights = $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

```
self.bias, self.weights = weights[0], weights[1:]
```

```
class MatrixLinearRegression:
```

```
def fit(self, X, y):
```

```
    X = np.insert(X, 0, 1, axis=1) # add ones vector
```

```
    XT_X_inv = np.linalg.inv(X.T @ X) # (X.T * X) ** (-1) inverse matrix
```

```
    weights = np.linalg.multi_dot([XT_X_inv, X.T, y]) # XT_X_inv * X.T * y
```

```
    self.bias, self.weights = weights[0], weights[1:]
```

```
def predict(self, X_test):
```

```
    return X_test @ self.weights + self.bias
```

$$\begin{bmatrix} 1 & 1 \end{bmatrix}^T$$

$$kX + b$$

$$b_0 + b_1x + b_2x_2 \dots$$

$$\begin{bmatrix} 1 & 1 & 2 \end{bmatrix}$$

$$\hat{y} = X \cdot k + b$$

$$\hat{y} = x \cdot 1 + 1 \quad \text{z.m.g.}$$

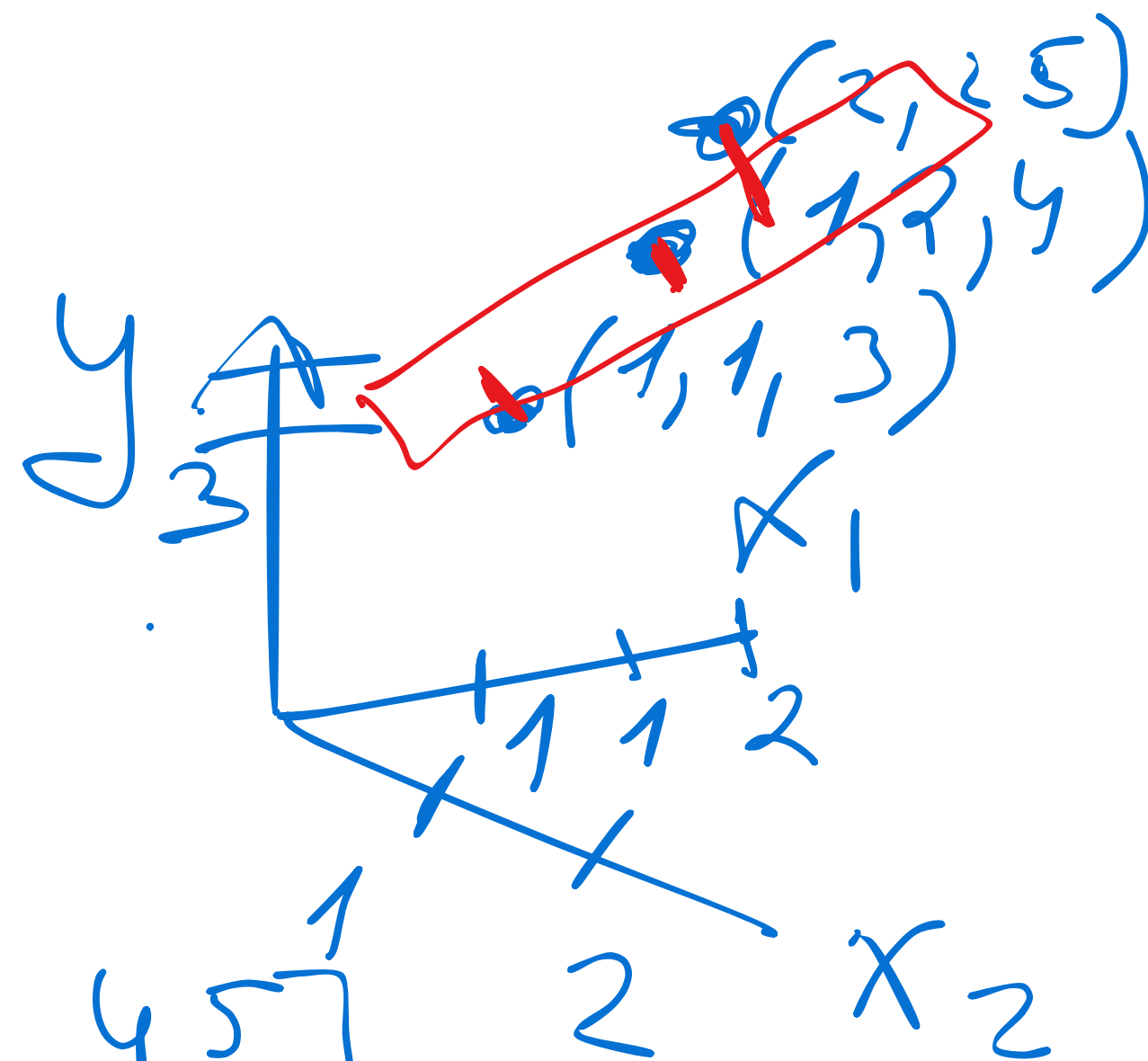
$$X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \end{bmatrix} \quad 3 \times 3$$

No	x_1	x_2	y
1	1	1	3
2	1	2	4
3	2	2	5

$$y = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \end{bmatrix}$$

$$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 4 & 5 \\ 4 & 6 & 7 \\ 5 & 7 & 9 \end{bmatrix}$$



```
weights = np.linalg.multi_dot([XT_X_inv, X.T, y])
```

$$X^T y \rightarrow \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \rightarrow \begin{bmatrix} 3+4+5 \\ 3+4+10 \\ 3+8+10 \end{bmatrix} \begin{matrix} 3 \times 1 \\ \\ \end{matrix} = \begin{bmatrix} 12 \\ 17 \\ 21 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 4 & 5 \\ 4 & 6 & 7 \\ 5 & 7 & 9 \end{bmatrix} - \begin{matrix} \downarrow \\ |A| \end{matrix} \cdot A^T \begin{bmatrix} + & - & + \\ - & + & - \\ + & - & + \end{bmatrix}$$

$$\det(A) = +3 \cdot \begin{vmatrix} 6 & 7 \\ 7 & 9 \end{vmatrix} - 4 \cdot \begin{vmatrix} 4 & 7 \\ 5 & 9 \end{vmatrix} + 5 \cdot \begin{vmatrix} 4 & 6 \\ 5 & 7 \end{vmatrix}$$

$$3 \cdot 5 - 4 - 10 = 15 - 14 = 1 \neq 0$$

$$\text{cof}(A) =$$

миноры кофакторы

$$A_{11} = + \cdot \begin{vmatrix} 6 & 7 \\ 7 & 9 \end{vmatrix} = 5$$

$$A_{12} = - \begin{vmatrix} 4 & 7 \\ 5 & 9 \end{vmatrix} = -1$$

$$A_{13} = + \begin{vmatrix} 4 & 6 \\ 5 & 7 \end{vmatrix} = -2$$

$$A_{21} = - \begin{vmatrix} 6 & 9 \\ 7 & 5 \end{vmatrix} = -1$$

$$A_{22} = + \begin{vmatrix} 4 & 9 \\ 5 & 5 \end{vmatrix} = 2$$

$$A_{23} = - \begin{vmatrix} 4 & 5 \\ 5 & 7 \end{vmatrix} = -1$$

$$A_{31} = + \begin{vmatrix} 4 & 7 \\ 6 & 7 \end{vmatrix} = -2$$

$$A_{32} = - \begin{vmatrix} 2 & 4 \\ 5 & 7 \end{vmatrix} = -1$$

$$A_{33} = + \begin{vmatrix} 3 & 4 \\ 6 & 6 \end{vmatrix} = 2$$

$$\text{cof}(A) = \begin{vmatrix} 5 & -1 & -2 \\ -1 & 2 & -1 \\ -2 & -1 & 2 \end{vmatrix}^T$$

$$A^{-1} = \frac{1}{\det A} \cdot \text{cof}(A)^T$$

$$\text{cof}(A)^T = \text{adj}(A)$$

сумма

компоненты

$$\begin{vmatrix} 5 & -1 & -2 \\ -1 & 2 & -1 \\ -2 & -1 & 2 \end{vmatrix} \cdot \begin{bmatrix} 12 \\ 17 \\ 21 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

1- compute $\Rightarrow 5 \cdot 12 + (-1) \cdot 17 + (-2) \cdot 21 = 1$

2- compute $\Rightarrow -1 \cdot 12 + 2 \cdot 17 - 1 \cdot 21 = -12 + 34 - 21 = 1$

3- compute $\Rightarrow -2 \cdot (12 + (-1) \cdot 17 + 2 \cdot 21) = 1$

① $\approx \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

$$y = w_0 + w_1 x_1 + w_2 x_2$$

$$= 1 + 1 \cdot x_1 + 1 \cdot x_2$$

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 3 & -1 & 1 \\ 4 & 0 & 1 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 3 & -1 & 1 \\ 4 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} & 0 \cdot (-1) \cdot 1 + 1 \cdot 3 \cdot 0 + \\ & 2 \cdot 1 \cdot 4 = 8 - 2 = 6 \\ & - 1 \cdot (-1) \cdot 4 = 1 \cdot 0 \cdot 0 = 6 \end{aligned}$$

$$A = \begin{bmatrix} 0 & 2 & 1 & 0 & 2 \\ 3 & -1 & 1 & 3 & -1 \\ 4 & 0 & 1 & 4 & 0 \end{bmatrix}$$

$$\det(A) = 6$$

$$\begin{aligned} & 0 \cdot (-1) \cdot 1 + 2 \cdot 1 \cdot 4 + 1 \cdot 3 \cdot 0 = 8 \\ & 4 \cdot (-1) \cdot (1) + 0 \cdot 1 \cdot 0 + 1 \cdot 3 \cdot 2 = 2 \end{aligned}$$

`X` — список списков чисел (матрица признаков, **регрессоры**),

`y` — список чисел (вектор **ответов**),

возвращает: список коэффициентов (вектор **theta**).

python

Copy

Edit

```
X = np.array(X)
```

◆ Преобразуем входной список `X` в **NumPy-массив** (удобно для матричных операций).

python

Copy

Edit

```
y = np.array(y).reshape(-1, 1)
```

◆ Преобразуем список `y` в NumPy-массив и делаем из него **вектор-столбец** (размерность $n \times 1$)

`reshape(-1, 1)` означает: сколько строк — неважно, но 1 столбец.

python

Copy

```
theta = np.linalg.inv(X_transpose.dot(X)).dot(X_transpose).dot(y)
```

◆ Это и есть **решение нормального уравнения**:

$$\theta = (X^T X)^{-1} X^T y$$

- `X_transpose.dot(X)` — это $X^T X$,
- `np.linalg.inv(...)` — берём **обратную матрицу**,
- далее умножаем на $X^T y$.

✚ В результате получаем вектор **коэффициентов линейной регрессии** — θ , где:

- первый элемент — свободный член (если в `X` есть столбец из единиц),
- остальные — веса регрессоров.

python

Copy

```
theta = np.round(theta, 4).flatten().tolist()
```

◆ Округляем коэффициенты до **4 знаков после запятой**,

◆ `flatten()` — превращаем в одномерный массив,

◆ `tolist()` — в обычный **список Python**.

метод `dot` в NumPy не всегда означает скалярное произведение — он работает по-разному в зависимости от формы (размерности) аргументов.

Случай	Что делает <code>dot</code>
Вектор • Вектор (1D)	Скалярное произведение
Матрица × Вектор	Матричное умножение
Матрица × Матрица	Матричное умножение

```
theta = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
```

Все аргументы — **матрицы**, а не одномерные векторы, так что:

- `X.T.dot(X)` — это **матричное произведение**: $X^T X$
- `.dot(X.T)` — ещё одно **матричное произведение**
- `.dot(y)` — произведение матрицы на вектор-столбец

Ранг матрицы меньше числа её ^(признаков) столбцов

Ранг матрицы меньше числа её столбцов (признаков), это означает, что не все столбцы (признаки) в этой матрице являются линейно независимыми.

Ранг матрицы – это максимальное количество линейно независимых столбцов (или, что эквивалентно, максимальное количество линейно независимых ³строк) в этой матрице.

$$X_3 = X_1 + X_2$$

	X_1	X_2	X_3	
Чел 1	[170, 60, 230]			// F1, F2, F3 для человека 1
Чел 2	[180, 75, 255]			// F1, F2, F3 для человека 2
Чел 3	[165, 55, 220]			// F1, F2, F3 для человека 3
Чел 4	[190, 90, 280]			// F1, F2, F3 для человека 4

Лекция 4

$$X_3 = \overset{1}{\lambda} \cdot X_1 + \overset{1}{\lambda} \cdot X_2 \quad \lambda \in \mathbb{R}$$

Ранг 2 < 3

Матрица координатности — представляет собой
координатность

координатность
координатность

признаки и многообразие

$$A = \begin{bmatrix} x_1 & x_2 \\ 14 & 28 \\ 28 & 56 \end{bmatrix}$$

$$x_2 = x_1 \cdot 2$$

$$= 14 \cdot 56 - 28 \cdot 28 = 0$$

$$\frac{1}{|A|} \cdot A^T \cdot \det(A) = 0$$

• INV

`np.linalg.matrix_rank(X)` и `np.linalg.det(X.T @ X)`

`correlation_matrix = income.corr()` корреляция между столбцами, между фичами

$$\theta = (X^T X)^{-1} X^T y$$

$$X^T X \theta = X^T y$$

$$\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$\begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

1.

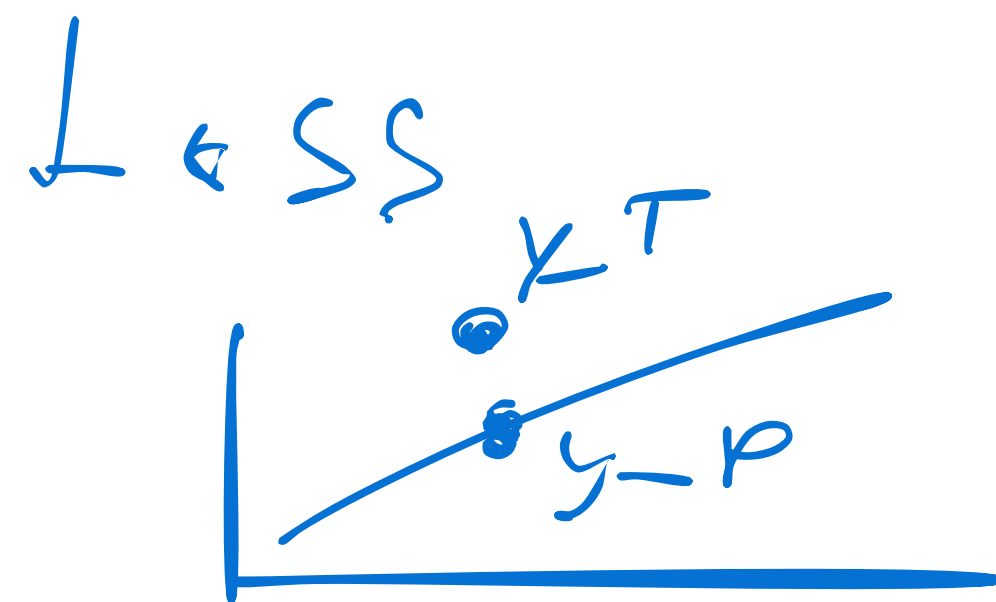
$$y = X\theta + \epsilon$$

$$\begin{bmatrix} x_0 & x_1 & \dots & x_n \\ x_0 & x_1 & \dots & x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_0 & x_1 & \dots & x_n \end{bmatrix}$$

$$h = \begin{bmatrix} 1 & x_1 & x_2 \\ 1 & x_1 & x_2 \\ 1 & x_1 & x_2 \\ 1 & x_1 & x_2 \end{bmatrix} \times \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \varepsilon_4 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}$$

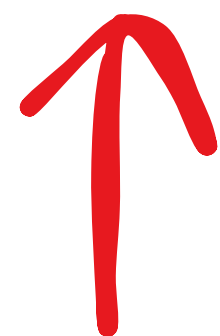
Функция потерь

$$\varepsilon = X \theta - y$$



Loss L_2

SSE $= \sum (y - X\theta)^2$



MSE $= \frac{1}{2n} \sum (y - \theta X)^2$

MHIK

$$\Theta = (X^T X)^{-1} X^T Y$$

$$\varepsilon = Y - \hat{y} = y - X\Theta$$

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - X_i^T \Theta)^2$$

$$\varepsilon = y - X\theta$$

сумму квадратов всех элементов этого вектора

$$SSE = \varepsilon^T \varepsilon = \overset{n \times 1}{(y - X\theta)}^T \overset{1 \times n}{(y - X\theta)}$$

$$(y - X\theta)^T (y - X\theta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Матричная форма суммы квадратов ошибок

$$(y - X\theta)^T (y - X\theta)$$

1. *klagraft* / *разнос*

$$y^T y - y^T (X\theta) - (X\theta)^T y + (X\theta)^T (X\theta)$$

$$A = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$b = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

$$A^T b = b^T A$$

$$A^T b = [1 \ 2] \begin{bmatrix} 3 \\ 4 \end{bmatrix} = 11$$

$$b^T A = [3 \ 4] \begin{bmatrix} 1 \\ 2 \end{bmatrix} = 11$$

$$y^T y - \underbrace{(X\theta)^T y}_{\text{red bracket}} - \underbrace{(X\theta)^T y}_{\text{red bracket}} + (X\theta)^T (X\theta)$$

$$y^T y - 2 \underbrace{(X\theta)^T y}_{\text{red bracket}} + (X\theta)^T (X\theta)$$

$$\underbrace{(A \ B)^T}_{\text{red bracket}} = B^T A^T$$

$$y^T y - 2 X^T \theta^T y + X^T \theta^T X \theta$$

$$y^T y - 2\theta^T X^T y + \theta^T X^T X \theta$$

$$\nabla J(\theta) (y^T y - 2\theta^T X^T y + \theta^T X^T X \theta)$$

$$\nabla J(\theta) = \frac{d y^T y}{d \theta} = 0 \quad 2X^T X \theta$$

$$\nabla J(\theta) = \frac{d(-2\theta^T X^T y)}{d \theta} = -2X^T y$$

$2A\theta^*$
 $X = X^T$
 симметрич

$$-2 \underbrace{\Theta^T}_{m \times n} \underbrace{X^T Y}_{m \times 1} \approx -2 X^T Y$$

$\begin{bmatrix} \end{bmatrix} \begin{bmatrix} \end{bmatrix} = m \times 1$

$\swarrow$
 $R^{m \times 1}$

$$\Theta^T (R^{m \times 1}) \rightarrow 1 \times 1 \Rightarrow \text{scalar}$$

$$- 2 \Theta^T X^T y = a^T \Theta \Rightarrow 1 \times 1 = \text{zero}$$

$\Rightarrow$ лекция - линейка

1. св-во транспонирования

$$(\Theta^T (X^T y))^T = (X^T y)^T \Theta =$$

$\square \square \square \Rightarrow |X|$
 $m \times n \rightarrow n \times m$
 $n \times 1 - a \rightarrow a^T \Theta$

$m \times 1$

$$a^T \theta \approx \nabla_{\theta} (a^T \theta) \approx$$

$$a \approx \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix}$$

$$\theta \approx \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

$$a^T \theta = a_1 \theta_1 + a_2 \theta_2 \dots a_n \theta_n$$

$$\nabla_{\theta} (a^T \theta) \approx \begin{bmatrix} \frac{\partial (a^T \theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial (a^T \theta)}{\partial \theta_n} \end{bmatrix}$$

$$\frac{\partial}{\partial \theta_1} (a_1 \theta_1' + \cancel{a_2 \theta_2} \dots) = a_1$$

$$\frac{\partial}{\partial \theta_2} \Rightarrow a_2$$

$$\frac{\partial}{\partial \theta_h} \Rightarrow a_h$$

$$\nabla_{\theta} (a^T \theta) = \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} = a$$

$$\theta^T X^T X \theta$$

$$X \sim \theta$$

$$A = X^T X$$

$$\frac{d(X^T A X)}{dX} = 2AX$$

$$2 \times 5 \times \theta$$

$$X^T X - \text{current type 2}$$

$$X_2 \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} X_2^T \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

$$\begin{bmatrix} X_1 & X_2 \\ 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}^T$$

$$1 \cdot 1 + 3 \cdot 3 + 5 \cdot 5 = 35$$

$$1 \cdot 2 + 3 \cdot 4 + 5 \cdot 6 = 44$$

$$X^T X = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} 35 & 44 \\ 44 & 56 \end{bmatrix}$$

$$-2X^T y = -2X^T X \Theta \quad | -1$$

$$2X^T y \rightarrow 2X^T X \Theta \quad | 2$$

$$X^T y \approx X^T X \Theta$$

— транспонировать уравнение

$$\Theta X^T X \approx X^T y$$

$$| (X^T X)^{-1}$$

$$\ominus \left| \frac{X^T X \cdot (X^T X)^{-1}}{1} \right. \approx (X^T X)^{-1} X^T Y$$

$$\ominus \rightarrow (X^T X)^{-1} X^T Y$$



$$\Theta = (X^T X)^{-1} X^T$$

CNAY

$$A\Theta \approx b$$

i

$$AX \approx$$

$n \times k$

$$\begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

$$\approx \begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix}$$

$$\begin{bmatrix} a_1 \\ \vdots \\ a_k \end{bmatrix}$$

$n \times k$

linear regression problem

$$A X = b$$

$$\begin{bmatrix} a_1 & a_2 & \dots & a_k \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}^{n \times k}$$

$$A = X^T X$$

$$\Theta = (X^T X)^{-1} X^T y$$

$$b = X^T y$$

~~Θ~~ X = vector parameters

$$AX = b \mid A^{-1} \rightarrow \text{symmetric bagpacker sampling}$$

$$A^{-1}AX = A^{-1}b$$

I eg. mapping

$$\theta = A^{-1}b$$

$$I\theta = \theta$$

$$X = A^{-1}b$$

Решить систему уравнений

$$Ax = b \quad | \quad A^{-1}$$

$$\cancel{A}^{-1} \cancel{A} x = \cancel{A}^{-1} b$$

$$x = A^{-1} b$$

Решить систему

$$3\theta_0 + 6\theta_1 = 6$$

$$6\theta_0 + 14\theta_1 = 14$$

$$u_3 = 1$$

$$\theta_0 = 2 - 2\theta_1$$

$$6(2 - 2\theta_1) + 14\theta_1 = 14$$

$$12 - 12\theta_1 + 14\theta_1 = 14$$

x_1	x_2	y
3	6	6
6	14	14

A b

$$12 + 2\theta_1 = 14$$

$$2\theta_1 = 14 - 12$$

$$2\theta_1 = 2 \quad | :2$$

$$\boxed{\theta_1 = 1}$$

$$Ax = b$$

$$\theta_0 = 2 - 2\theta_1$$

$$\theta_0 = 2 - 2(1)$$

$$\theta_0 = 2 - 2$$

$$\theta_0 = 0$$

$$x = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 14 \end{bmatrix}$$

$\begin{matrix} A & x & b \end{matrix}$

CAY

$$\begin{bmatrix} 6 \\ 14 \end{bmatrix}$$

$$A \cdot \vec{x} = \vec{b}$$

Метод Гаусса

$$\begin{bmatrix} x_1 & x_2 & x_3 \\ 2 & 1 & 8 \\ 1 & 3 & 13 \end{bmatrix}$$

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

$$\vec{b} = \begin{bmatrix} 8 \\ 13 \end{bmatrix}$$

Реш. вручную

$$\left[\begin{array}{cc|c} 2 & 1 & 8 \\ 1 & 3 & 13 \end{array} \right]$$

$$R_2 \leftarrow R_2 - \frac{1}{2} R_1$$

$$\left[\begin{array}{cc|c} 2 & 1 & 8 \\ 0 & 2.5 & 9 \end{array} \right]$$

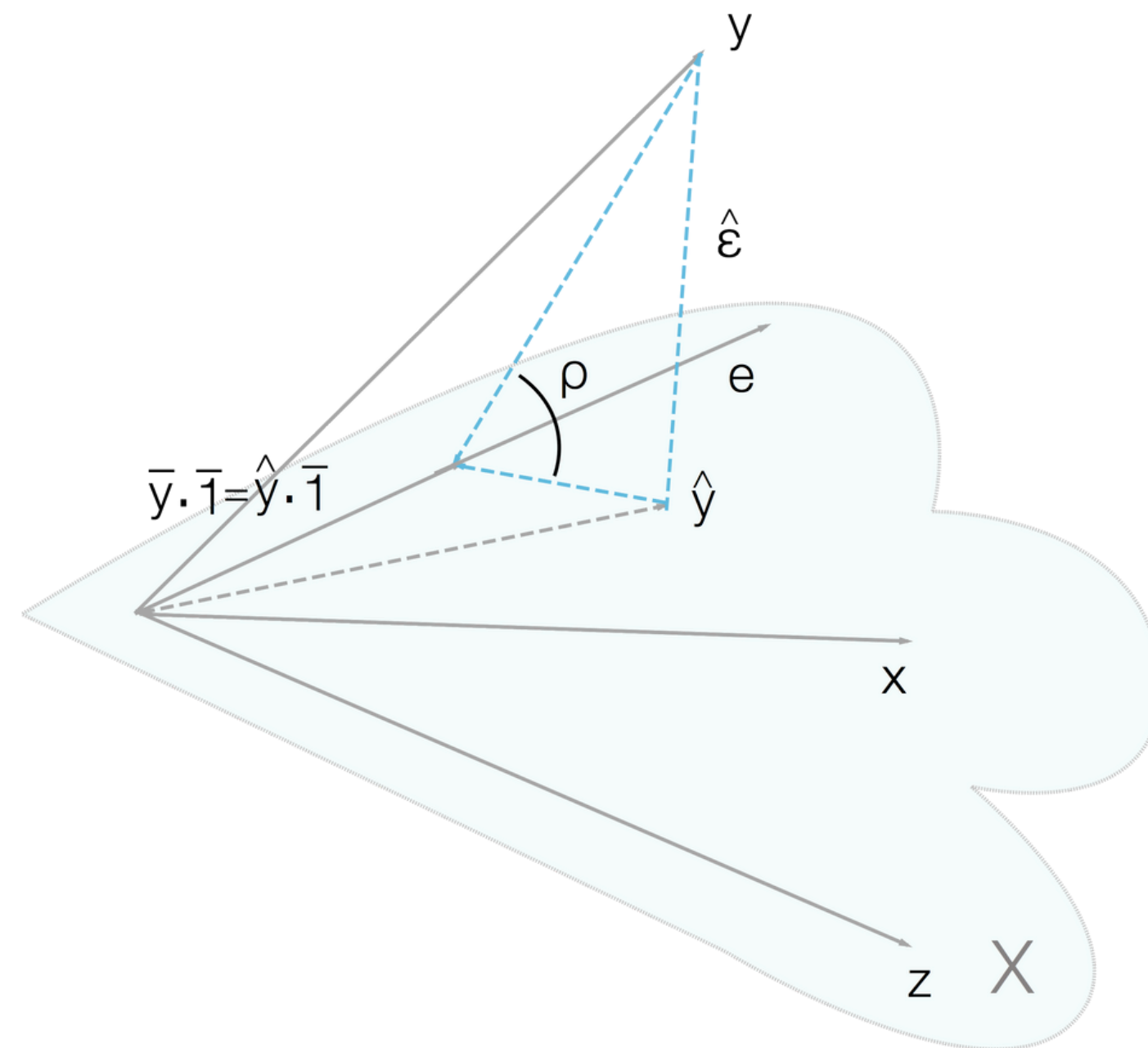
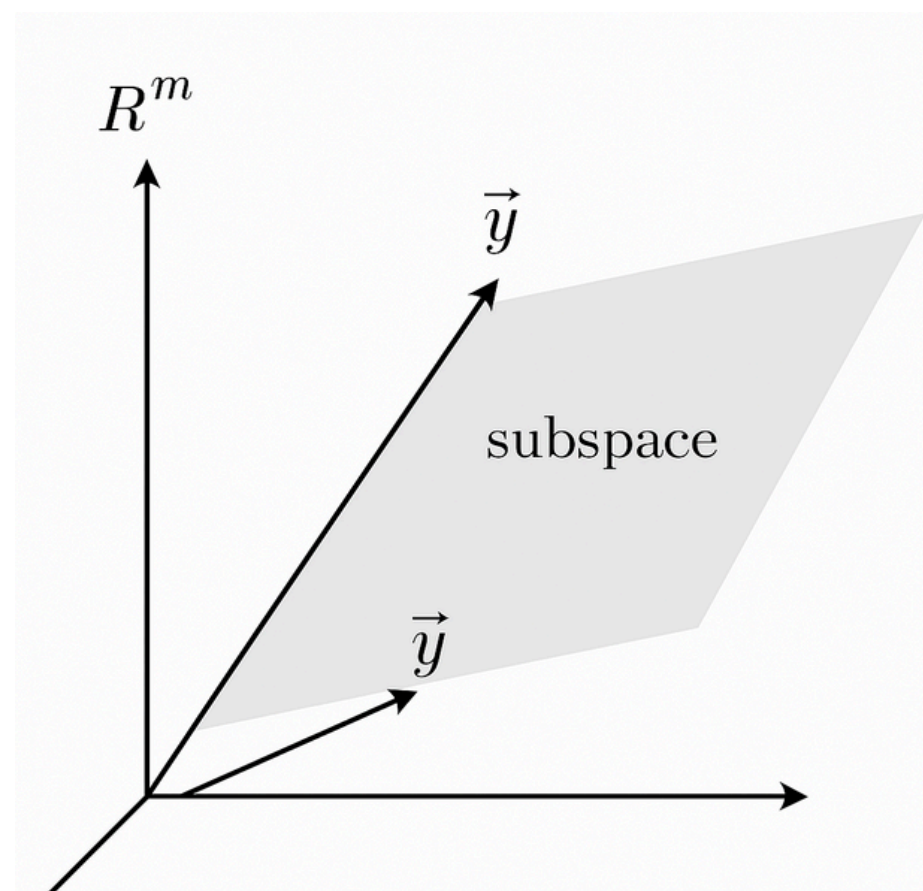
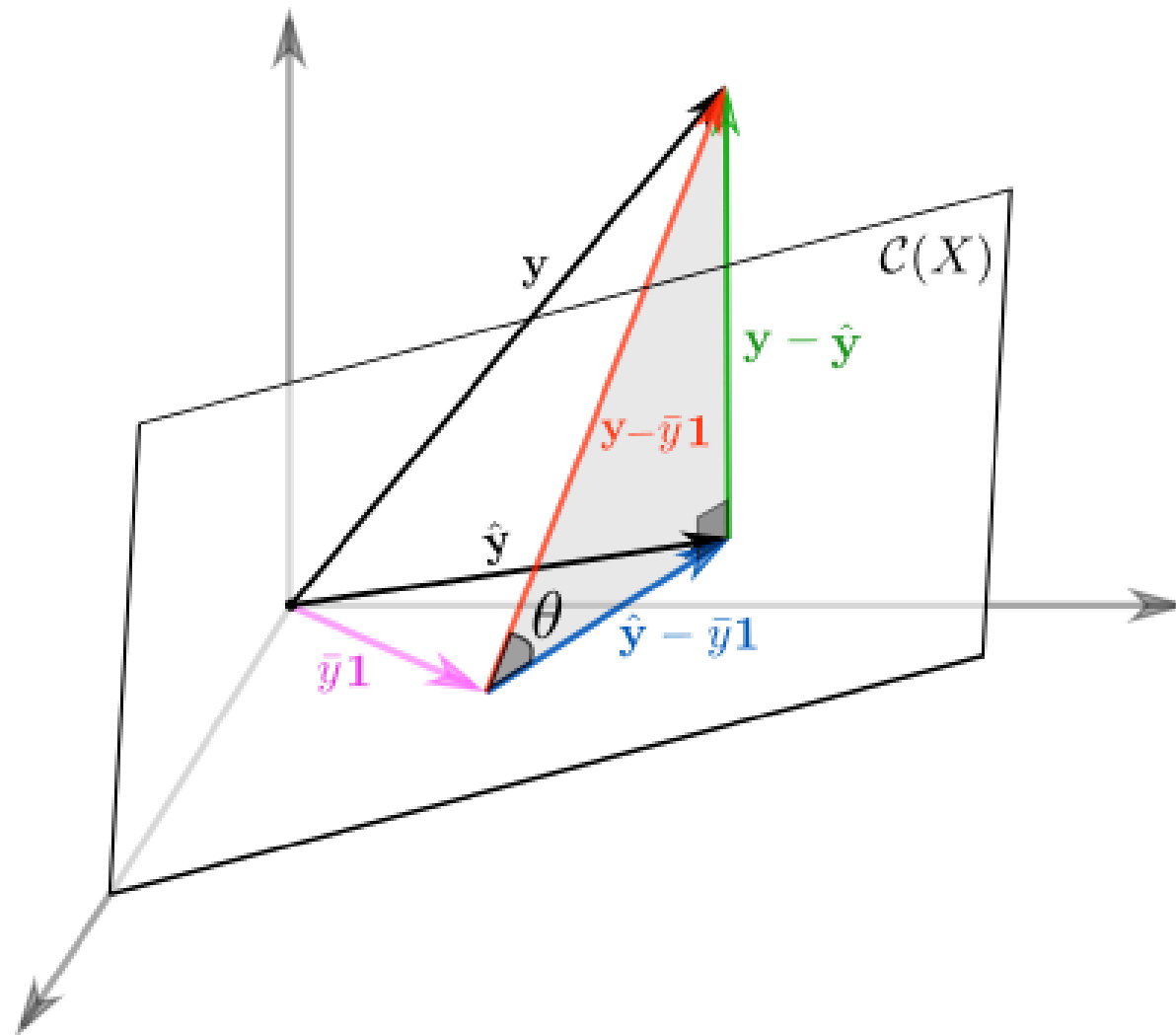
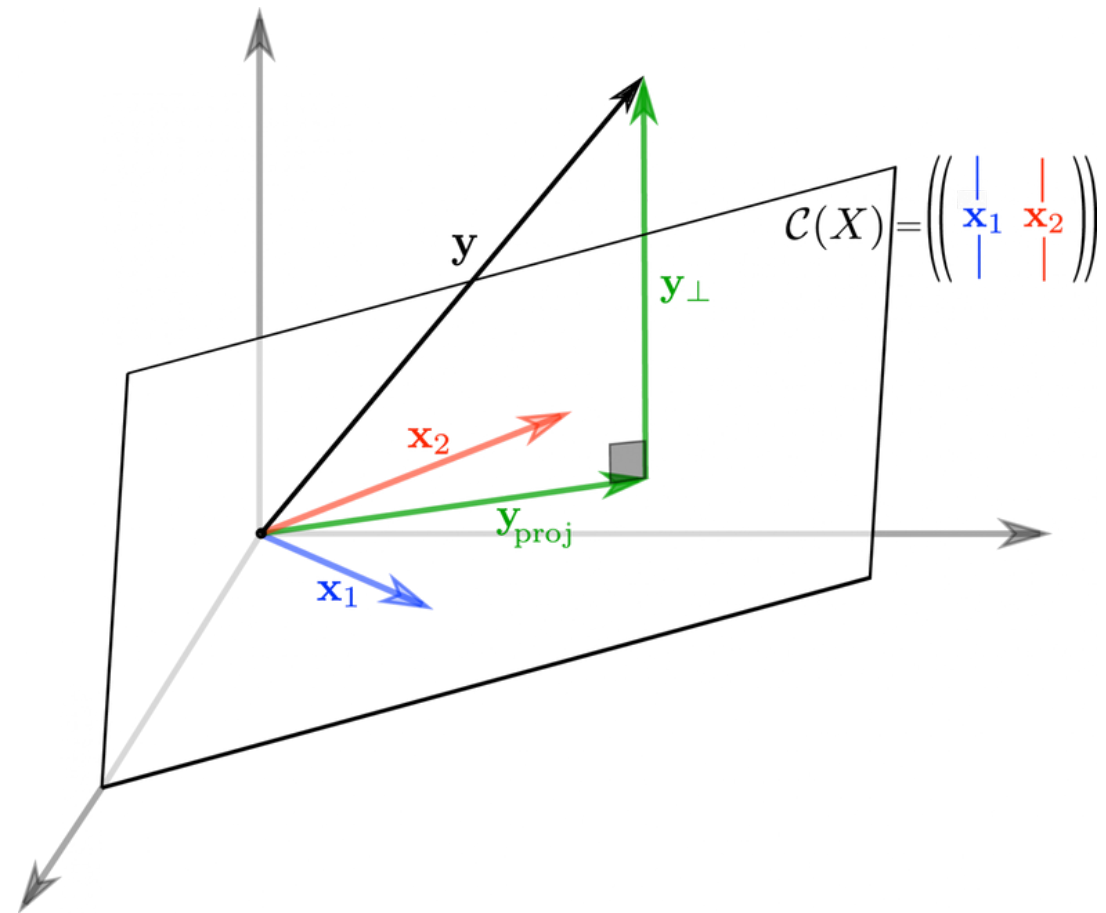
$$\begin{array}{ccc} 1 & 3 & 13 \\ 1 & 0.5 & 4 \\ \hline 0 & 2.5 & 9 \end{array}$$

$\Theta \delta_f$ амбур $\alpha \log$

$$\cancel{\Theta_1}^{\circ} + \Theta_2 2.5 = 9 \quad | \quad 2.5$$

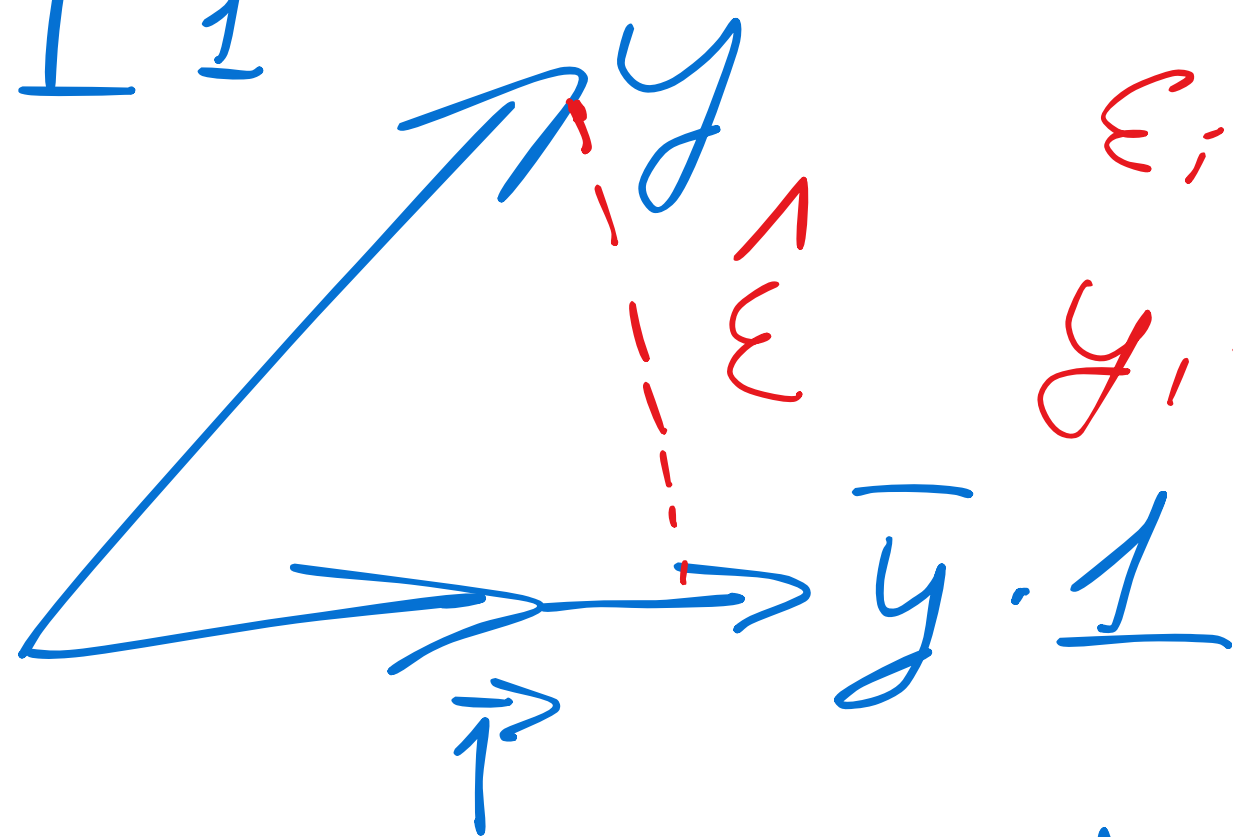
$$\Theta_2 = 3.6$$

$$2\Theta_1 + 3.6 = 8 \Rightarrow 2\Theta_1 = 4.4 \quad | \quad 2 \quad \Theta_1 = 2.2$$



$$y = \beta + \epsilon$$

$$\hat{\epsilon} \perp \vec{1}$$



$$\epsilon_i = y_i - \hat{y}$$

$$y_i = \hat{y} + \epsilon_i$$

$$y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

$$\vec{1} = \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix}$$

$$\hat{\beta} = \bar{y}$$

$$y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

$$\hat{y}_i = \bar{y}$$

$$= \begin{pmatrix} \bar{y} \\ \vdots \\ \bar{y} \end{pmatrix}$$

$$\hat{y} \rightarrow \bar{y} \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

$$y_i = \beta_1 + \beta_2 x_1 + \beta_3 x_2 + \varepsilon$$

$$\begin{pmatrix} y_i \\ \varepsilon \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & x_1 \\ 1 & x_2 \end{pmatrix} \Rightarrow \begin{pmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{pmatrix}$$

$$\vec{y} = \beta_1 \cdot \vec{1} + \beta_2 x_1 + \beta_3 x_2$$

$$x^T e = 0$$

$\text{Col}(X)$

1
 ε

$$C(X) =$$

$\text{Col}(X)$

$$\begin{pmatrix} 1 & 1 \\ X_1 & X_2 \\ 1 & 1 \end{pmatrix}$$

$\tau, \bar{\tau}, g$



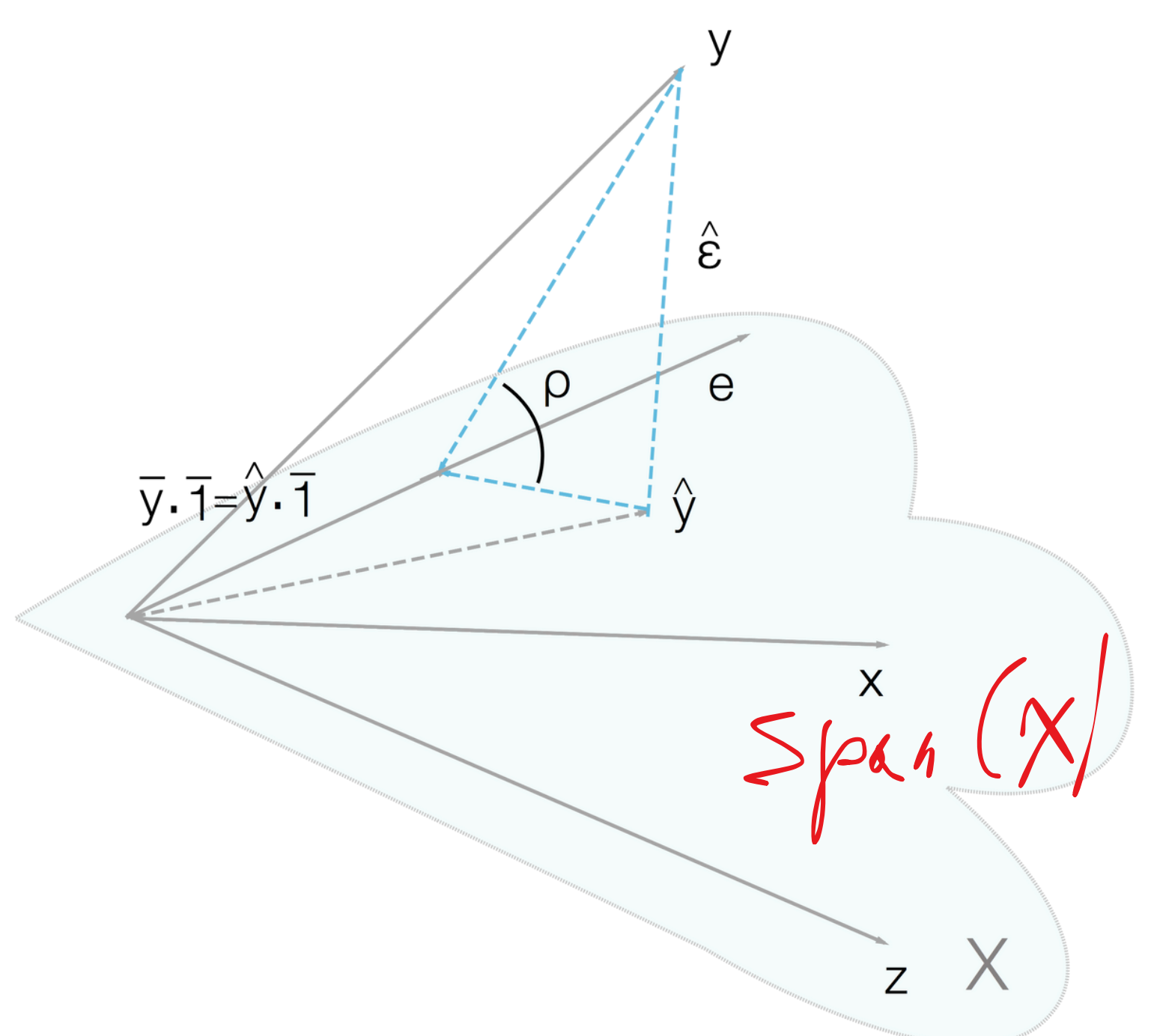
1

$$y = \beta_0 \cdot 1 + \beta_1 x_1 + \beta_2 x_2$$

$$TSS = ESS + RSS$$

$$SST = SSR + SSE$$

SS E



$$\frac{ESS}{TSS} = \frac{b^2 C^2}{Ab^2} = \left(\frac{bC}{Ab} \right)^2$$

$$= (\cos \rho)^2$$